# Übungsblatt 1: Zahlendarstellung, Logik und Register

**Mikrocomputertechnik 1**

**Musterlösung**

## Aufgabe 1: Anzeigewerte eines Klimaschranks (10 Punkte)

Ein Klimaschrank für Materialprüfungen speichert alle Mess- und Stellwerte als Bitmuster; er begleitet Sie durch das ganze Blatt. Heiz- und Lüfterstufe reichen von 0 bis 255. In dieser Aufgabe sind alle Werte vorzeichenlos.

1. Ergänzen Sie die Tabelle für acht Bit. Die erste Zeile ist ein Beispiel. (6 P)

| Größe | Dezimal | Summe der Zweierpotenzen | Binär | Hex |
|---|---|---|---|---|
| Luftfeuchte in % | 42 | $32 + 8 + 2$ | `0010 1010` | `0x2A` |
| Luftfeuchte in % | 75 | | | |
| Heizstufe | | | | `0xB6` |
| Lüfterstufe | | | `0110 1101` | |

2. Der Laufzeitzähler ist 16 Bit breit und zählt Minuten. Er zeigt `0x05A0`. Geben Sie das 16-Bit-Muster und die Laufzeit in Minuten und in Stunden an. Tipp: Jede Hexziffer ergibt genau vier Bit. (2 P)
3. Bis zu welchem Wert kann dieser Zähler zählen, dezimal und hexadezimal? Reicht er für einen Prüflauf über 30 Tage? (2 P)

**Musterlösung**

| Größe | Dezimal | Summe der Zweierpotenzen | Binär | Hex |
|---|---|---|---|---|
| Luftfeuchte in % | 75 | $64 + 8 + 2 + 1$ | `0100 1011` | `0x4B` |
| Heizstufe | 182 | $128 + 32 + 16 + 4 + 2$ | `1011 0110` | `0xB6` |
| Lüfterstufe | 109 | $64 + 32 + 8 + 4 + 1$ | `0110 1101` | `0x6D` |

1. Je Zeile 2 P, anteilig.
2. `0000 0101 1010 0000` (1 P). $1024+256+128+32 = 1440$ Minuten, also 24 Stunden (1 P). Ebenso richtig über die Hexstellen: $5 \cdot 256 + 10 \cdot 16 = 1440$.
3. $2^{16} - 1 = 65\,535$ = `0xFFFF` (1 P). 30 Tage sind $30 \cdot 24 \cdot 60 = 43\,200$ Minuten. Das ist kleiner als 65 535, der Zähler reicht (1 P).

## Aufgabe 2: Temperaturen mit Vorzeichen (12 Punkte)

Der Klimaschrank arbeitet von −40 °C bis +120 °C. Temperaturen speichert er in ganzen Grad Celsius als 8-Bit-Zweierkomplement.

1. Welcher Wertebereich ist mit acht Bit im Zweierkomplement darstellbar? Passt der Arbeitsbereich hinein? (1 P)
2. Ergänzen Sie die Tabelle. Die erste Zeile ist ein Beispiel. (4 P)

| Temperatur | positives Muster | invertiert | plus eins | Hex |
|---|---|---|---|---|
| −5 °C | `0000 0101` | `1111 1010` | `1111 1011` | `0xFB` |
| −18 °C | | | | |
| −40 °C | | | | |

3. Die Anzeige meldet beim Muster $\texttt{0xE7}$ eine Temperatur von 231 °C. Welche Lesart hat sie verwendet? Welche Temperatur ist tatsächlich gemeint? (2 P)
4. Der Prozessor des Schranks rechnet mit 32-Bit-Registern. Erweitern Sie $\texttt{0xE7}$ so auf 32 Bit, dass die Temperatur erhalten bleibt. Welchen Wert hätte stattdessen die Nullerweiterung? (2 P)
5. Ein Prüfprogramm erhöht den Sollwert schrittweise um 20 °C ($\texttt{0x14}$), testweise auch über den Arbeitsbereich hinaus. Ergänzen Sie die Tabelle für acht gespeicherte Bits. (2 P)

| Rechnung | Ergebnisbits | Übertrag | Signed-Überlauf | signed gelesen |
|---|---|---|---|---|
| `0x14 + 0x14` ($20 + 20$) | `0x28` | 0 | nein | +40 |
| `0xEE + 0x14` ($-18 + 20$) | | | | |
| `0x6E + 0x14` ($110 + 20$) | | | | |

6. Nach dem Schritt von 110 °C auf 130 °C zeigt die Anzeige −126 °C. Erklären Sie dieses Symptom. Mit welchem 16-Bit-Muster ließe sich +130 °C darstellen? (1 P)

**Musterlösung**

1. $-128$ bis $+127$. Der Arbeitsbereich −40 °C bis +120 °C passt hinein (1 P).

| Temperatur | positives Muster | invertiert | plus eins | Hex |
|---|---|---|---|---|
| −18 °C | `0001 0010` | `1110 1101` | `1110 1110` | `0xEE` |
| −40 °C | `0010 1000` | `1101 0111` | `1101 1000` | `0xD8` |

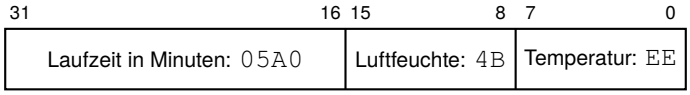
2. Je Zeile 2 P, anteilig.
3. Die Anzeige hat das Muster vorzeichenlos gelesen: $128 + 64 + 32 + 4 + 2 + 1 = 231$ (1 P). Im Zweierkomplement zählt das höchste Bit negativ: $-128 + 64 + 32 + 4 + 2 + 1 = -25$, gemeint sind −25 °C (1 P). Kontrolle: $231 - 256 = -25$.
4. Vorzeichenerweiterung: $\texttt{0xFFFFFFE7}$, weiterhin −25 (1 P). Die Nullerweiterung $\texttt{0x000000E7}$ hätte den Wert 231 (1 P).

| Rechnung | Ergebnisbits | Übertrag | Signed-Überlauf | signed gelesen |
|---|---|---|---|---|
| `0xEE + 0x14` ($-18+20$) | `0x02` | 1 | nein | +2 |
| `0x6E + 0x14` ($110+20$) | `0x82` | 0 | ja | −126 |

5. Je Zeile 1 P. Der Übertrag in der ersten Zeile ist kein Fehler: $-18+20=2$ liegt im Bereich.
6. 130 liegt über 127 und ist mit acht Bit im Zweierkomplement nicht darstellbar. Die Bits `1000 0010` werden als $-128+2=-126$ gelesen (0,5 P). Mit 16 Bit: `0x0082` (0,5 P).

## Aufgabe 3: Messprotokoll im Speicher (10 Punkte)

Der Schrank legt jede Messung als 32-Bit-Protokollwort im byteadressierten Speicher ab, wie in der Kursumgebung Little Endian. Nach 24 Stunden bei 75 % Luftfeuchte und −18 °C entsteht mit den Werten aus Aufgabe 1 und 2 das Wort `0x05A04BEE`. Es beginnt bei Adresse `0x2008`.




1. Ergänzen Sie die Bytes an den aufsteigenden Adressen. Die erste Zeile ist ein Beispiel. (3 P)

| Adresse | Little Endian | Big Endian |
|---|---|---|
| 0x2008 | EE | 05 |
| 0x2009 | | |
| 0x200A | | |
| 0x200B | | |

2. An welcher Adresse liegt im Kursspeicher die Temperatur, an welcher das höchstwertige Byte des Worts? (1 P)
3. Ein Auswerteprogramm auf einem PC liest die vier Bytes ab `0x2008` und zeigt das Wort `0xEE4BA005` an. Welche Byteordnung hat es angenommen? Welche Temperatur meldet es deshalb? (2 P)
4. Der nächste Eintrag folgt direkt auf den ersten. An welcher Adresse beginnt er, und ist sie für ein Wort natürlich ausgerichtet? Wäre `0x200A` natürlich ausgerichtet? Tipp: Für die Teilbarkeit durch vier genügt die letzte Hexziffer. (2 P)
5. Ein eigener Protokollspeicher des Schranks verwendet 16-Bit-Byteadressen. Wie groß ist sein Adressraum in KiB, und wie viele Protokollwörter lassen sich darin höchstens adressieren? Muss dafür so viel Speicher eingebaut sein? (2 P)

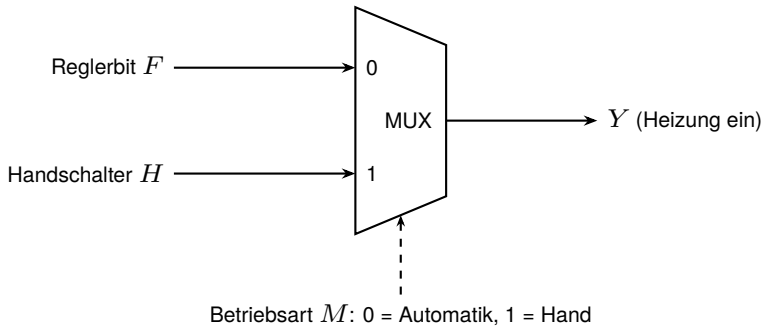
**Musterlösung**

| Adresse | Little Endian | Big Endian |
|---|---|---|
| 0x2009 | 4B | A0 |
| 0x200A | A0 | 4B |
| 0x200B | 05 | EE |

1. Je Zeile 1 P.
2. Die Temperatur `EE` liegt an `0x2008`, das höchstwertige Byte `05` an `0x200B` (1 P).
3. Big Endian: Das Programm liest das Byte an der kleinsten Adresse als höchstwertiges (1 P). In den Bits 7 bis 0 steht dann `0x05`, es meldet +5 °C statt −18 °C (1 P).
4. `0x2008` + 4 = `0x200C`. Die letzte Hexziffer C = 12 ist durch vier teilbar, die Adresse ist natürlich ausgerichtet (1 P). `0x200A` lässt bei Division durch vier den Rest 2 und ist nicht ausgerichtet (1 P).
5. $2^{16} = 65\,536$ Byte = 64 KiB, also höchstens $65\,536/4 = 16\,384$ Protokollwörter (1 P). Nein: Der Adressraum sagt nichts darüber, wie viel Speicher eingebaut ist (1 P).

## Aufgabe 4: Hand- oder Automatikbetrieb (10 Punkte)

Die Heizung des Schranks läuft bei $Y = 1$. Im Automatikbetrieb ($M = 0$) bestimmt das Reglerbit $F$ den Ausgang, im Handbetrieb ($M = 1$) der Handschalter $H$. Das ist ein 2:1-Multiplexer mit dem Auswahlsignal $M$.




1. Ergänzen Sie die Wahrheitstabelle. Zwei Zeilen sind vorgegeben. (3 P)

| $M$ | $H$ | $F$ | gewählte Quelle | $Y$ |
|---|---|---|---|---|
| 0 | 0 | 0 | | |
| 0 | 0 | 1 | $F$ | 1 |
| 0 | 1 | 0 | | |
| 0 | 1 | 1 | | |
| 1 | 0 | 0 | | |
| 1 | 0 | 1 | | |
| 1 | 1 | 0 | $H$ | 1 |
| 1 | 1 | 1 | | |

2. Geben Sie $Y$ als Gleichung mit UND, ODER und Negation an. Tipp: Je Betriebsart ein UND-Term. (2 P)
3. Im Automatikbetrieb stellt jemand den Handschalter auf Ein, der Regler meldet $F = 0$. Heizt der Schrank? Welcher Teilterm Ihrer Gleichung gibt hier seine Quelle frei, und welchen Wert hat er? (1 P)
4. Mit demselben $M$ wählt die Steuerung den 8-Bit-Sollwert: im Automatikbetrieb $\texttt{0xD8}$ (−40 °C) aus dem Prüfprogramm, im Handbetrieb $\texttt{0x14}$ (+20 °C) vom Drehknopf. Welches Wort liegt bei $M = 0$, welches bei $M = 1$ am Ausgang? Wie viele Ein-Bit-Multiplexer sind nötig, und welches Signal teilen sie? (2 P)
5. Nach einer Reparatur heizt der Schrank im Handbetrieb, obwohl der Handschalter auf Aus steht; nur der Regler meldet $F = 1$. Beim Aufbau wurde die Negation an das falsche UND-Gatter angeschlossen: $Y' = (M \wedge F) \vee (\overline{M} \wedge H)$. Bestimmen Sie $Y'$ für $(M, H, F) = (1, 0, 1)$ und $(0, 1, 0)$. Was beobachtet man deshalb im Automatikbetrieb? (2 P)

**Musterlösung**

| $M$ | $H$ | $F$ | gewählte Quelle | $Y$ |
|---|---|---|---|---|
| 0 | 0 | 0 | $F$ | 0 |
| 0 | 1 | 0 | $F$ | 0 |
| 0 | 1 | 1 | $F$ | 1 |
| 1 | 0 | 0 | $H$ | 0 |
| 1 | 0 | 1 | $H$ | 0 |
| 1 | 1 | 1 | $H$ | 1 |

1. Je Zeile 0,5 P.
2. $Y = (\overline{M} \wedge F) \vee (M \wedge H)$. Je Term 1 P.
3. Nein, $Y = 0$. Freigegeben ist der Term $\overline{M} \wedge F$; er hat den Wert $1 \wedge 0 = 0$. Der Handschalter wirkt nur bei $M = 1$ (1 P).
4. $M = 0$: `0xD8`, $M = 1$: `0x14` (1 P). Acht Ein-Bit-Multiplexer, alle mit demselben Auswahlsignal $M$ (1 P).
5. $(1, 0, 1)$: $Y' = (1 \wedge 1) \vee (0 \wedge 0) = 1$, richtig wäre 0. $(0, 1, 0)$: $Y' = (0 \wedge 0) \vee (1 \wedge 1) = 1$, richtig wäre 0 (1 P). Die Betriebsarten sind vertauscht: Im Automatikbetrieb folgt die Heizung dem Handschalter statt dem Regler (1 P).

## Aufgabe 5: Regelabweichung im Rechenwerk (8 Punkte)

Der Regler bildet die Regelabweichung $E = \text{Soll} - \text{Ist}$. Wir rechnen in einem verkleinerten Vier-Bit-Modell im Zweierkomplement (−8 bis 7) mit $A - B = A + \overline{B} + 1$: alle Bits von $B$ invertieren und den Startübertrag $C_0 = 1$ setzen. Die Nullerkennung $Z$ ist genau dann eins, wenn alle vier Ergebnisbits null sind. Sie schaltet die LED „Sollwert erreicht“.

1. Ergänzen Sie Summenbit $S$ und Ausgangsübertrag $C_{\text{out}}$ eines Volladdierers. Die erste Zeile ist ein Beispiel. (2 P)

| $A$ | $B$ | $C_{\text{in}}$ | $S$ | $C_{\text{out}}$ |
|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 |
| 0 | 1 | 1 | | |
| 1 | 0 | 0 | | |

2. Soll = 2 (`0010`), Ist = 5 (`0101`). Ergänzen Sie die Übertragskette für $E = 2 - 5$. Stelle 0 ist vorgegeben. (2 P)

| Stelle $i$ | $A_i$ | $B_i$ | $\overline{B_i}$ | $C_i$ | $S_i$ | $C_{i+1}$ |
|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 | 1 | 0 |
| 1 | 1 | 0 | | | | |
| 2 | 0 | 1 | | | | |
| 3 | 0 | 0 | | | | |

3. Geben Sie $E$ als Bitmuster $S_3S_2S_1S_0$ und als vorzeichenbehafteten Wert an. Leuchtet die LED? (1 P)
4. Jetzt gilt Soll = Ist = 5. Bestimmen Sie die vier Ergebnisbits, den finalen Übertrag $C_4$ und $Z$. (2 P)
5. In einem Aufbau bleibt die LED bei Soll = Ist = 5 dunkel. Eine Messung zeigt $C_0 = 0$. Welches Ergebnis bildet das Rechenwerk, und warum bleibt die LED aus? (1 P)

**Musterlösung**

| $A$ | $B$ | $C_{\text{in}}$ | $S$ | $C_{\text{out}}$ |
|---|---|---|---|---|
| 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 |

1. Je Zeile 1 P. Zwei Einsen ergeben $2 = 0 + 2 \cdot 1$, eine Eins ergibt $1 = 1 + 2 \cdot 0$.

| Stelle $i$ | $A_i$ | $B_i$ | $\overline{B_i}$ | $C_i$ | $S_i$ | $C_{i+1}$ |
|---|---|---|---|---|---|---|
| 1 | 1 | 0 | 1 | 0 | 0 | 1 |
| 2 | 0 | 1 | 0 | 1 | 1 | 0 |
| 3 | 0 | 0 | 1 | 0 | 1 | 0 |

2. Je Zeile 2/3 P.

3. $E = 1101$, also $-8 + 4 + 1 = -3$: Der Istwert liegt drei Grad über dem Sollwert. $Z = 0$, die LED bleibt aus (1 P).
4. `0101 + 1010 + 1 = 1 0000`: Ergebnisbits `0000` (1 P), $C_4 = 1$ und $Z = 1$, die LED leuchtet (1 P). Der Übertrag $C_4$ ist hier kein Fehler.
5. `0101 + 1010 + 0 = 1111`, also $-1$. Nicht alle Bits sind null, daher $Z = 0$. Ohne Startübertrag bildet die Kette nur $A + \overline{B} = A - B - 1$ (1 P).

## Aufgabe 6: Messwerte im Register halten (10 Punkte)

Ein 8-Bit-Register hält den angezeigten Istwert. Der Sensor legt fortlaufend einen neuen Wert an $D$. Das Register übernimmt $D$ nur an einer steigenden Flanke mit $WE = 1$; alle Zeitbedingungen sind erfüllt.

1. Anfangs gilt $Q = 20$. Ergänzen Sie $Q$ nach jedem Ereignis. Die erste Zeile ist ein Beispiel. (4 P)

| Ereignis | $D$ | $WE$ | $Q$ danach |
|---|---|---|---|
| Steigende Flanke | 21 | 1 | 21 |
| Keine Flanke | 23 | 1 | |
| Steigende Flanke | 23 | 0 | |
| Fallende Flanke | 24 | 1 | |
| Steigende Flanke | 24 | 1 | |

2. In einem anderen Gerät bleibt die Anzeige auf 21 °C stehen, obwohl der Sensor längst 24 liefert. Der Takt läuft mit regelmäßigen steigenden Flanken. Welches Signal prüfen Sie zuerst, und welchen Wert erwarten Sie dort? (1 P)
3. Die Schreibfreigabe ist wie in der Vorlesung mit einem Multiplexer vor dem Register gebaut. Welchen Wert gibt dieser Multiplexer in der dritten Tabellenzeile ($D = 23$, $WE = 0$) an das Register weiter? (1 P)

Der Prozessor des Schranks hält Istwert und Korrektur in Registern: `x9` = 21, `x10` = 3, `x11` = 0. Bei `add` steht das Ziel zuerst, danach folgen die beiden Quellen. Die Befehle laufen nacheinander.

4. Ergänzen Sie die Tabelle. Die erste Zeile ist ein Beispiel. (3 P)

| Befehl | gelesene Werte | Summe | Wirkung |
|---|---|---|---|
| `add x11, x9, x10` | 21 und 3 | 24 | `x11` = 24 |
| `add x0, x11, x9` | | | |
| `add x9, x9, x10` | | | |
| `add x10, x9, x11` | | | |

5. Geben Sie `x0`, `x9`, `x10` und `x11` nach dem letzten Befehl an. (1 P)

**Musterlösung**

1. $Q$ in Tabellenreihenfolge: 21 (Beispiel), 21, 21, 21, 24. Je Zeile 1 P. Ohne Flanke und bei $WE = 0$ hält das Register; die fallende Flanke ist keine aktive Flanke.
2. Die Schreibfreigabe $WE$. Sie ist vermutlich dauerhaft 0 (1 P).
3. Den rückgeführten alten Wert $Q = 21$. An der Flanke übernimmt das Register denselben Wert, der Zustand bleibt 21 (1 P).

| Befehl | gelesene Werte | Summe | Wirkung |
|---|---|---|---|
| `add x0, x11, x9` | 24 und 21 | 45 | keine, `x0` verwirft den Wert |
| `add x9, x9, x10` | 21 und 3 | 24 | `x9` = 24 |
| `add x10, x9, x11` | 24 und 24 | 48 | `x10` = 48 |

4. Je Zeile 1 P. Der dritte Befehl liest den alten Wert 21 von `x9` und schreibt danach 24. Der vierte Befehl sieht bereits diesen neuen Wert.
5. `x0` = 0, `x9` = 24, `x10` = 48, `x11` = 24 (1 P).

## Aufgabe 7: Die Registerspur in Ripes prüfen (optional)

Das folgende Programm führt die vier Befehle aus Aufgabe 6 aus. Davor setzen zwei `addi`-Befehle (Addition einer Konstanten) `x9` = 21 und `x10` = 3. `j inspect` ist eine Dauerschleife, an der Sie den Zustand prüfen.

```
.text
.globl main
main:
    addi x9, x0, 21
    addi x10, x0, 3
    add  x11, x9, x10
    add  x0, x11, x9
    add  x9, x9, x10
    add  x10, x9, x11
inspect:
    j inspect
```

1. Öffnen Sie Ripes, installiert oder im Browser unter ripes.dk. Geben Sie das Programm im Tab *Editor* links ein; Ripes übersetzt es automatisch. Wählen Sie über *Select Processor* den *RISC-V Single Cycle Processor* für RV32I ohne zusätzliche Erweiterungen unter *ISA Exts.* Protokollieren Sie Ripes-Version und Modell.
2. Notieren Sie Ihre Vorhersage aus Aufgabe 6 für `x0`, `x9`, `x10` und `x11`, zusätzlich als Hexwerte.
3. Klicken Sie *Reset* und dann einzeln *Clock*; *Reverse* nimmt einen Takt zurück. Verfolgen Sie im Tab *Processor* in der Ansicht *Registers*, wann sich `x9`, `x10` und `x11` ändern. Ändert sich beim Befehl mit dem Ziel `x0` ein Register? Hören Sie auf, sobald das Programm bei `inspect` kreist, und vergleichen Sie die Endwerte mit Ihrer Vorhersage. Achten Sie darauf, ob die Werte hexadezimal oder dezimal angezeigt werden.
4. Ändern Sie die Korrektur in der zweiten Zeile auf −2 (`addi x10, x0, -2`). Sagen Sie das 32-Bit-Muster von `x10` nach diesem Befehl und die Endwerte der vier Register voraus. Prüfen Sie dann im Simulator.

Register, die das Programm nicht verwendet, können vom Simulator vorbelegt sein; sie spielen hier keine Rolle.

#### Musterlösung

Erwartung bei `inspect`: `x0` = 0, `x9` = 24 = `0x18`, `x10` = 48 = `0x30`, `x11` = 24 = `0x18`. Nach den beiden `addi` gilt `x9` = 21 = `0x15` und `x10` = 3. Beim Befehl mit dem Ziel `x0` ändert sich kein allgemeines Register, nur der PC schreitet fort. Die Schleife bei `inspect` verändert die Datenregister nicht mehr.

Mit der Korrektur −2 enthält `x10` nach dem zweiten `addi` das 32-Bit-Zweierkomplement `0xFFFFFFFE`. Danach: `x11` = 19 = `0x13`, die Summe 40 an `x0` wird verworfen, `x9` = 19 = `0x13`, `x10` = 38 = `0x26`.

Das sind Modellvorhersagen; Version, Modell und tatsächliche Anzeige gehören ins Protokoll.